DEBRECENI EGYETEM
INFORMATIKAI KAR

SZAKDOLGOZAT

Tappmancs

Webalkalmazás állatmenhelyek és örökbefogadni kívánók részére

Témavezető:

Major Sándor Roland

Készítette:

Dobrosi Gergő

Programtervező Informatikus

Debrecen

2026

Tartalomjegyzék

1. Bevezetés 5

1.1 A témaválasztás indoklása 5

1.2 A téma elméleti és gyakorlati jelentősége 5

1.3 Célkitűzések 5

1.4 Alkalmazott módszerek 6

2. Technikai áttekintés 7

2.1 Platform és keretrendszerek 7

2.2 A projekt elérhetősége 9

2.3 A projekt futtatása 9

2.4 A tervezés folyamata 10

Ütemterv 10

Funkcionális terv és use case diagram 11

Adatbázisterv 12

Látványtervek 14

3. A megvalósítás részletei 15

A rész: Backend (fejlesztői dokumentáció) 15

3.1 Architektúra 15

3.2 A repó felépítése 16

3.3 Adatmodell 17

3.4 Szerepkörök és jogosultságkezelés 20

3.5 Livewire komponensek 21

3.6 Képfeltöltés 22

3.7 A fő funkciók a kódban 23
Többlépéses regisztráció 23
Keresés és szűrés 23
Munkatárskezelés 23
Üzenetküldés a menhelynek 23
Validáció és lokalizáció 23
3.8 Ütemezett feladatok 24
3.9 Tesztelés 24
Unit és feature tesztek 24
Végponttól végpontig tartó (e2e) tesztek 25
3.10 Kihelyezés és üzemeltetés 26
Az éles szerver 26
A hosting kiválasztása 26
Automatikus kihelyezés 27
3.11 AI-val gyorsított fejlesztés 28
B rész: Frontend (felhasználói útmutató) 28
3.12 Böngészés látogatóként 28
3.13 Kisállat- és menhelyadatlap 30
3.14 Regisztráció és bejelentkezés 32
3.15 Örökbefogadóként: üzenet a menhelynek 33
3.16 Menhelytulajdonosként 34
3.17 Munkatársként 38
3.18 Fiókbeállítások 38
3.19 Mobilnézet 39

4. Összefoglalás 40
4.1 Az eredmények 40
4.2 Mit tanultam a projektből? 40
4.3 Új technológiák és módszerek 40
4.4 Mi volt nehéz és mi volt egyszerű? 41
4.5 Meg nem valósult célkitűzések 41
4.6 Továbbfejlesztési lehetőségek 41
Irodalomjegyzék 42
Internetes források 42
Függelék 45
F.1 Tesztfiókok 45
F.2 Az adatbázis teljes egyed-kapcsolat diagramja 45
F.3 A projekt időrendje 47
Köszönetnyilvánítás 48

1. Bevezetés

1.1 A témaválasztás indoklása

Évente sok kutya és macska kerül menhelyre vagy állatvédő egyesülethez. Az örökbefogadásra váró állatokról szóló információ ugyanakkor nagyon szétszórt, és a menhelyek nagy része kizárólag közösségi oldalakon, csoportokban teszi közzé a kisállatait. Aki örökbe szeretne fogadni, annak sok különböző helyen kell keresnie, a menhelyek pedig nehezen tudják naprakészen tartani, melyik kisállat érhető még el, melyik foglalt, és melyik talált már gazdira.

A témát azért választottam, mert egy valós, mindennapi problémára szerettem volna megoldást találni, és közben egy webalkalmazás elkészítésén keresztül szakmailag is fejlődni. Egy menhelykezelő felület ehhez ideális terület, mert több szerepkört és jogosultságkezelést, képfeltöltést, keresést és szűrést, valamint a felhasználók közötti kommunikációt is magában foglal, a végeredményt pedig valódi felhasználók is kipróbálhatják.

1.2 A téma elméleti és gyakorlati jelentősége

A gyakorlatban a Tappmancs egy helyre gyűjti össze az állatmenhelyeket és azokat, akik örökbe szeretnének fogadni. A menhelyek saját adatlapot kapnak, fel tudják tölteni a kisállataikat, és munkatársakat is felvehetnek, akik szintén kezelhetik a kisállatokat. Az érdeklődők név, település, faj, nem és státusz szerint tudnak keresni és szűrni, és ha megtetszik nekik egy kisállat, közvetlenül az oldalon keresztül üzenhetnek a menhelynek.

Szakmai oldalról egy webalkalmazás teljes elkészítését mutatom be a tervezéstől az éles működésig: az adatmodellt, a szerveroldali logikát, a jogosultságkezelést, a tesztelést és a kihelyezést. Kitérek arra is, hogyan használtam a fejlesztés során AI kódasszisztenst.

1.3 Célkitűzések

A célokat még a fejlesztés elején, 2024 novemberében egy ütemtervben és egy funkcionális tervben rögzítettem (lásd 2.4.). A projekt során végül ezek lettek a célkitűzéseim:

- Egy nyilvánosan elérhető webalkalmazás, amelyben a látogatók regisztráció nélkül böngészhetik a kisállatokat és a menhelyeket.
- Négy jól elkülönülő szerepkör: látogató, örökbefogadó, menhelytulajdonos és menhelyi munkatárs, mindegyik a saját jogosultságaival.
- Kisállat- és menhelylista kereséssel, szűréssel és képgalériával.
- Többlépéses regisztráció, amely a szerepkörtől függően vezeti végig a felhasználót.
- Munkatárskezelés: a tulajdonos e-mail cím alapján vehet fel munkatársat, aki onnantól a menhely kisállatait is kezelheti.
- Üzenetküldés: az örökbefogadó egy kisállatról üzenhet a menhelynek, a tulajdonos pedig listázhatja és törölheti az üzeneteket.
- Automatizált tesztelés unit, feature és végponttól végpontig (e2e) szinten.
- Éles üzemeltetés saját domainen, HTTPS-sel, automatikus kihelyezéssel, amely csak sikeres backend tesztfutás után módosítja a szervert.

1.4 Alkalmazott módszerek

A fejlesztést kisebb lépésekben végeztem 2024 decemberétől 2026 októberéig: először az adatmodell és a regisztráció készült el, utána a kisállatok és a menhelyek kezelése, végül a keresés, a munkatárskezelés, az üzenetküldés és az üzemeltetés.

A munkát agilis módszertan szerint, a szoftverfejlesztési életciklus (SDLC) lépéseit követve próbáltam szervezni, a fejlesztést sprintekre bontottam. Az egyes sprintek eredményeit (deliverable) a témavezetőmmel közösen néztük át és beszéltük meg.

A működést automatizált tesztek ellenőrzik: a backendet Pest unit és feature tesztek, a teljes felhasználói folyamatokat Playwright e2e tesztek. A kihelyezés automatikus, a GitHub Actions csak akkor frissíti az éles szervert, ha minden backend teszt sikeres.

A fejlesztéshez AI kódasszisztenst (Claude Code) is használtam, erről a 3.11. alfejezetben írok.

2. Technikai áttekintés

2.1 Platform és keretrendszerek

A Tappmancs egy szerveroldalon renderelt webalkalmazás, amely a Laravel keretrendszer modell-nézet-vezérlő (MVC) felépítésére épül, vagyis a böngésző minden oldalt kész HTML-ként kap meg [1]. Ahol az oldalnak újratöltés nélkül kell reagálnia, például az e-mail cím ellenőrzésénél vagy a képfeltöltésnél, Livewire komponenseket használtam [2], a kisebb böngészőoldali dolgokat (menük, galéria, felugró ablakok) pedig az Alpine.js kezeli [3].

Külön frontend alkalmazás (például React egy REST API-val) helyett azért döntöttem így, mert egyedül fejlesztettem: így egy kódbázist és egy kihelyezési folyamatot kell karbantartanom, és nem kell külön API-t terveznem.

Réteg	**Technológia**	**Szerepe a projektben**
Backend	PHP 8.5, Laravel 11	Útvonalak, vezérlők, adatbázis-kezelés, hitelesítés, jogosultságok, ütemezett feladatok
Hitelesítés	Laravel Breeze 2	A bejelentkezés, a regisztráció és a jelszókezelés kiinduló kódja
Dinamikus komponensek	Livewire 3, Alpine.js	Újratöltés nélküli űrlapok és felületi elemek
Felület	Blade, Tailwind CSS 3, Heroicons	Sablonok, reszponzív megjelenés, ikonok
Fordítás	Vite 6	A CSS és a JavaScript fordítása és csomagolása
Adatbázis	MySQL 8	Az alkalmazás összes adata
Backend tesztek	Pest 3, külön MySQL tesztadatbázis	Unit és feature tesztek

Réteg	Technológia	Szerepe a projektben
E2E tesztek	Playwright, TypeScript	Felhasználói folyamatok valódi böngészőben
Kódstílus	Laravel Pint, ESLint, Prettier	Egységes kódformázás
Szerver	DigitalOcean Droplet, Ubuntu 24.04, nginx, PHP-FPM, certbot	Éles üzemeltetés
CI/CD	GitHub Actions	Automatikus tesztelés és kihelyezés
Csomagkezelés	Composer, npm workspaces	PHP és JavaScript függőségek, monorepo
AI asszisztens	Claude Code	Gyorsított fejlesztés

1. táblázat: A projektben használt technológiák

A Laravel a legelterjedtebb PHP keretrendszer, és beépítve tartalmaz szinte mindent, ami a projekthez kellett: útvonalkezelést, Eloquent ORM-et, hitelesítést, jogosultságkezelést és ütemezőt. A nézetek a Blade sablonnyelvvel készültek [4].

A bejelentkezéshez és a regisztrációhoz kiindulópontnak a Laravel Breeze kezdőcsomagot használtam [5], amely legenerálja a hitelesítés alapjait (bejelentkezés, regisztráció, jelszó-visszaállítás). Ezt alakítottam át a saját igényeimre, a legtöbbet a regisztráción változtattam (lásd 3.7.).

A megjelenéshez a Tailwind CSS-t használtam, amelyben a stílusokat kis, egyetlen dologért felelős osztályokból lehet összerakni közvetlenül a sablonban [6]. A fájlokat a Vite fordítja le, és az éles fordításba csak a ténylegesen használt osztályok kerülnek be [7].

Adatbázisnak MySQL 8-at használok [8], a tesztek pedig egy külön MySQL tesztadatbázison futnak, így ugyanazon az adatbázismotoron ellenőrzöm a működést, amelyik élesben is fut. A backend teszteket a Pest keretrendszerrel írtam [9], az e2e teszteket pedig a Playwrighttal, amely egy valódi böngészőt vezérel [10].

Az éles környezet egy DigitalOcean Droplet, vagyis egy virtuális gép [11], nginx webszerverrel [12]. A kihelyezést a GitHub Actions végzi [13], a fejlesztéshez pedig a Claude Code AI kódasszisztenst is használtam [14].

2.2 A projekt elérhetősége

A teljes forráskód a GitHubon érhető el:

https://github.com/DobrosiGergo/tappmancs_szakdolgozat_24-25_26

Az alkalmazás élesben is fut, és bárki kipróbálhatja:

https://tappmancs-szakdolgozat.hu

Az éles oldal adatai minden hajnalban visszaállnak egy kiinduló állapotra (lásd 3.8.). A kipróbáláshoz használható tesztfiókok a Függelékben vannak.

2.3 A projekt futtatása

A repó egy npm workspaces monorepo [15], amely két csomagból áll: az app mappában a Laravel alkalmazás, az app-e2e mappában pedig a Playwright tesztek vannak. Helyi futtatáshoz a következők kellenek:

- PHP 8.2 vagy újabb
- Composer 2
- Node.js 20 vagy újabb
- MySQL 8 vagy újabb
- Git

A telepítés és az indítás lépései:

```
# 1. A kód letöltése
git clone https://github.com/DobrosiGergo/tappmancs_szakdolgozat_24-25_26.git
cd tappmancs_szakdolgozat_24-25_26

# 2. JavaScript függőségek (a gyökérben, mindkét csomaghoz)
npm install

# 3. PHP függőségek
```

```
cd app
composer install

# 4. Környezeti fájl és alkalmazáskulcs
cp .env.example .env
php artisan key:generate

# 5. Adatbázisok létrehozása (a második a teszteké)
mysql -u root -e 'create database tappmancs'
mysql -u root -e 'create database tappmancs_test'

# 6. Táblák és alapadatok
php artisan migrate --seed

# 7. Demo fiókok és demo menhely (nem kötelező)
php artisan demo:seed

# 8. A feltöltött képek kiszolgálása
php artisan storage:link

# 9. Indítás a repó gyökeréből
cd ..
npm run dev
```

Ezután az alkalmazás a http://localhost:8000 címen érhető el.

A backend tesztekhez a fenti második paranccsal létrehozott tappmancs_test adatbázis kell. A teszteket a gyökérből az npm run test (backend) és az npm run test:e2e (e2e) paranccsal lehet futtatni. Az e2e tesztek előtt telepíteni kell a böngészőket (npx playwright install chromium firefox).

2.4 A tervezés folyamata

A fejlesztés előtt, 2024 novemberében elkészítettem a projekt tervét: egy ütemtervet, a funkciók listáját, egy use case diagramot, az adatbázis tervét és a fő oldalak látványtervét. Ebben az alfejezetben ezeket hasonlítom össze azzal, ami végül elkészült.

Ütemterv

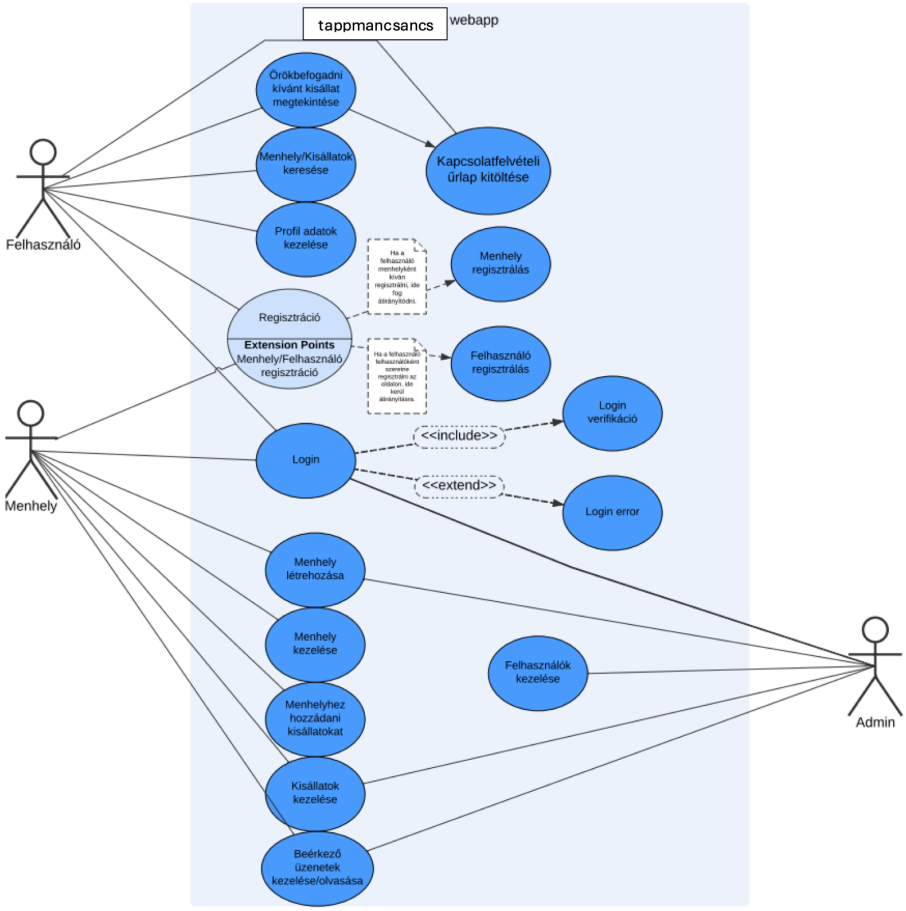
Az eredeti ütemtervben a munkát hat szakaszra bontottam:

- 2024. november: tervezés és előkészületek (célok, funkciók, adatbázis-szerkezet, a fejlesztési környezet és a Laravel telepítése)
- 2024. december: adatbázis és backend fejlesztés (migrációk, modellkapcsolatok, CRUD műveletek, üzenetküldés)
- 2025. január: frontend fejlesztés (Blade sablonok, regisztráció, profiloldal, keresés és szűrés, reszponzív megjelenés)
- 2025. február: üzenetkezelés és adminisztrációs felület (menhelyi üzenetkezelés, kommentek és kedvelések, moderálás)
- 2025. március: biztonság, tesztelés és dokumentáció, a szakdolgozat írásának kezdete
- 2025. április: utolsó simítások, a bemutató és a szakdolgozat befejezése

A megvalósítás végül tovább tartott, és 2026 októberében fejeződött be. A csúszás fő oka az volt, hogy közben teljes állásban kezdtem dolgozni tesztautomatizálási mérnökként (test automation engineer) az EPAM-nál, így a projektre jóval kevesebb időm maradt.

Funkcionális terv és use case diagram

A tervben szereplő funkciók: regisztráció és bejelentkezés, a menhelyek és a kisállatok kezelése, keresés, kapcsolatfelvétel a menhellyel, profilkezelés, adminisztrátori funkciók, és opcionálisan a munkatársak kezelése. Ezeket egy UML use case diagramon foglaltam össze három szereplővel: felhasználó, menhely és adminisztrátor.

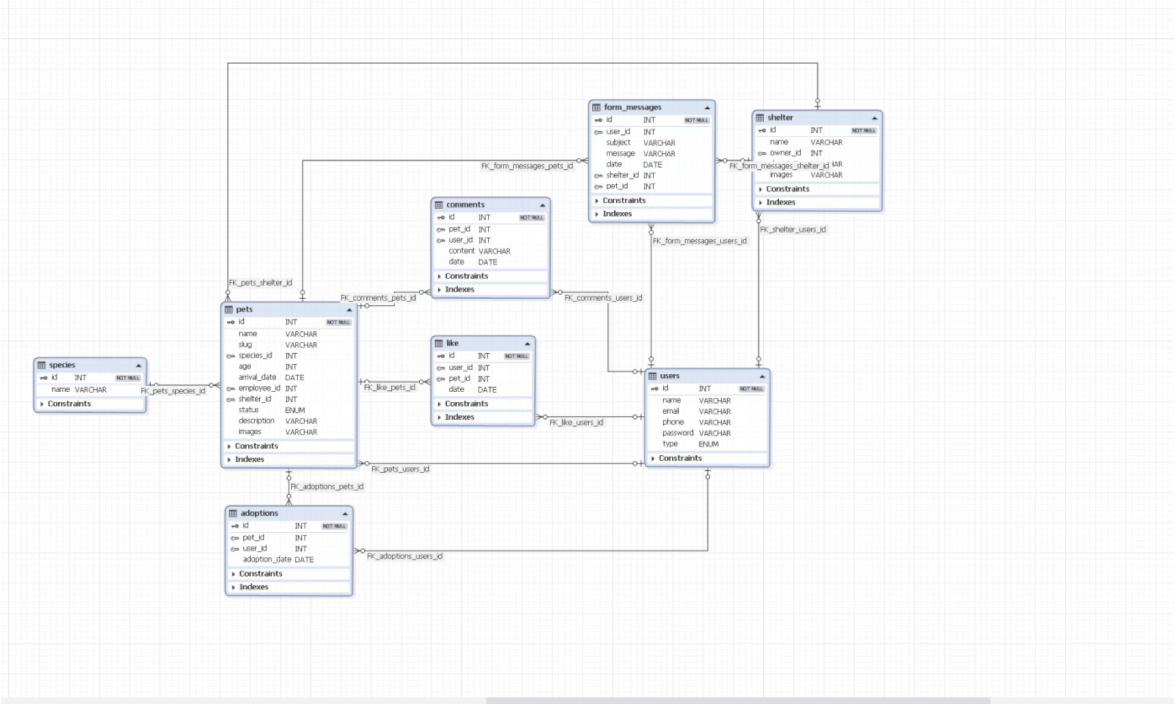


1. ábra: A rendszer use case diagramja (tervezési fázis, 2024. november)

Ehhez képest a felhasználó (a mai örökbefogadó) és a menhely minden esete megvalósult, az adminisztrátor szerepkör viszont nem (lásd 4.5.). A tervben kérdéses munkatárskezelés elkészült, ezért a „menhely” szereplő két szerepkörre vált szét: tulajdonosra és munkatársra.

Adatbázisterv

Az adatbázis első tervét szintén a tervezési fázisban készítettem el.



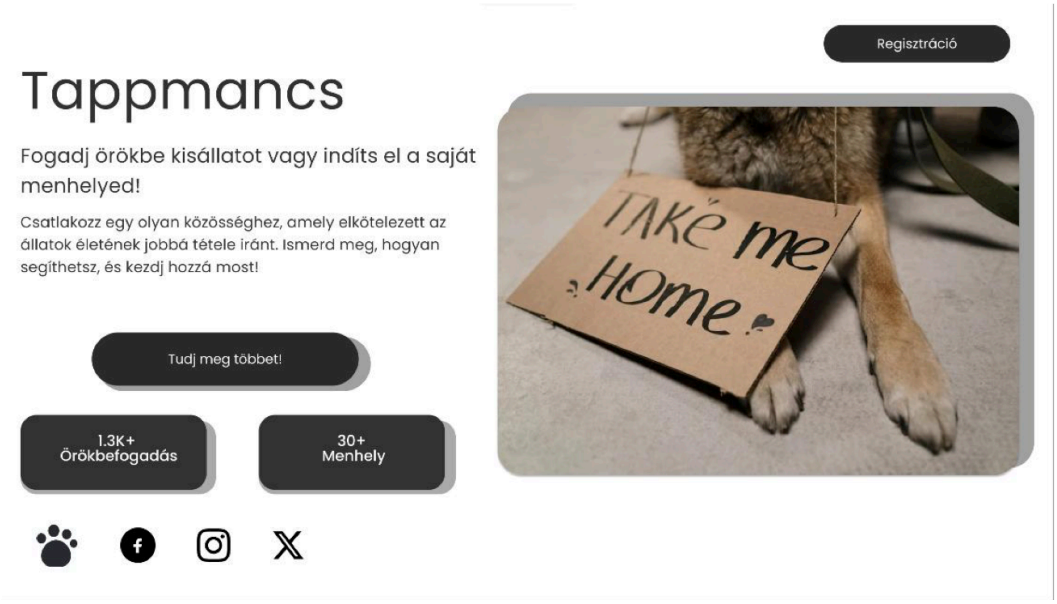
2. ábra: Az adatbázis első terve (tervezési fázis)

A végleges adatmodell (3.3.) a terv szerkezetét követi, de a fejlesztés közben több helyen változtattam rajta:

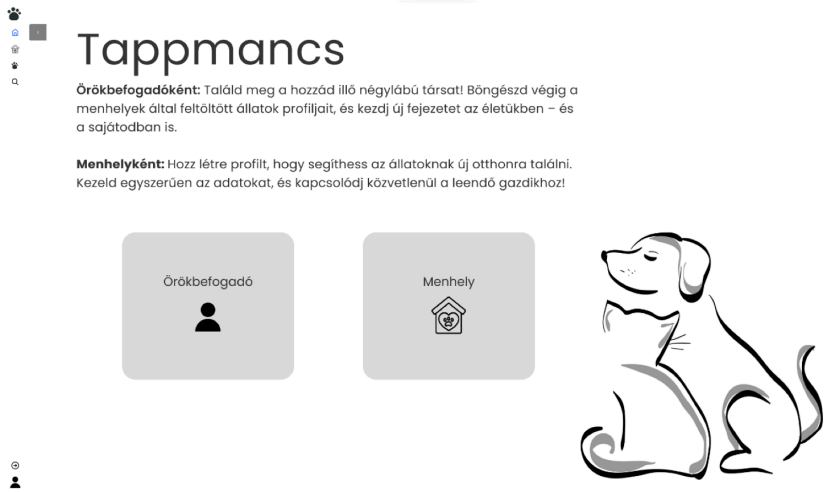
- Minden fő tábla uuid oszlopot kapott, mert az URL-ekben a sorszám helyett ezt használom.
- Külön breeds (fajták) tábla jött létre, mert a fajt és a fajtát egymástól függő listákból lehet kiválasztani.
- A kisállat életkora (age) helyett a születési dátumát (birth_date) tárolom, mert az életkor magától változik.
- A users tábla shelter_id mezőt kapott, ezen keresztül kapcsolódik a munkatárs a menhelyhez. Ez a munkatárskezelés miatt vált szükségessé.
- A kommentek, a kedvelések és az örökbefogadások táblái a terv szerint elkészültek, de a felület még nem épül rájuk.

Látványtervek

A fő oldalak látványterve is ekkor készült (nyitóoldal, listák, a regisztráció lépései, profiloldal), ezek közül kettőt mutatok be.



3. ábra: A nyitóoldal látványterve



4. ábra: A regisztráció szerepkörválasztó lépésének látványterve

A végleges felület megtartotta a tervek szerkezetét. A legnagyobb különbség, hogy a bal oldali ikonos sáv helyett felső menüsor lett, amely mobilon egy gombbal nyitható le.

3. A megvalósítás részletei

Az A rész fejlesztői dokumentáció az alkalmazás felépítéséről, a B rész felhasználói útmutató a felülethez.

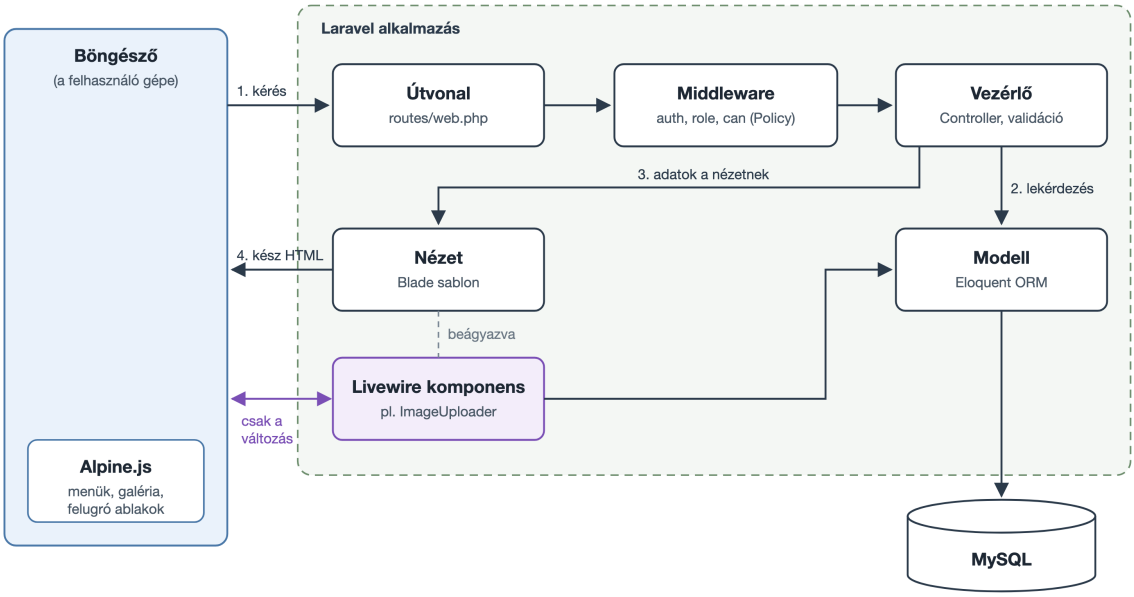
A rész: Backend (fejlesztői dokumentáció)

3.1 Architektúra

Az alkalmazás a Laravel MVC felépítését követi. Egy beérkező HTTP kérés a következő úton megy végig:

- **Útvonal (route):** a routes/web.php és a routes/auth.php fájl határozza meg, melyik URL melyik vezérlő melyik metódusához tartozik [16].
- **Middleware:** a kérés a vezérlő előtt szűrőkön halad át. Ezek ellenőrzik például, hogy a felhasználó be van-e jelentkezve, és megfelelő-e a szerepköre [17].
- **Vezérlő (controller):** validálja a bemenetet, meghívja a jogosultság-ellenőrzést, és az Eloquent modelleken keresztül olvassa vagy módosítja az adatokat.
- **Modell:** az Eloquent ORM osztályai, amelyek egy-egy adatbázistáblát képviselnek, és leírják a táblák közötti kapcsolatokat [18].
- **Nézet (view):** a Blade sablon, amely a vezérlőtől kapott adatokból előállítja a válasz HTML-jét.

Ahol egy oldalrésznek újratöltés nélkül kell reagálnia, a nézetbe ágyazott Livewire komponens a háttérben elküldi az eseményt a szervernek, és csak a megváltozott HTML-részt cseréli ki. A csak böngészőben futó dolgokat az Alpine.js kezeli.



5. ábra: Az alkalmazás architektúrája és egy kérés útja

3.2 A repó felépítése

A repó egy npm workspaces monorepo: a gyökérben lévő package.json fogja össze a két csomagot, így egy npm install mindkettő függőségeit telepíti, és a fő parancsokat a gyökérből lehet futtatni.

Mappa	Tartalom
app/	A Laravel alkalmazás
app-e2e/	A Playwright e2e tesztek
.github/workflows/	A GitHub Actions kihelyezési folyamat

2. táblázat: A repó fő mappái

Mappa	Tartalom
app/Console/Commands	Saját Artisan parancsok: demo:reset, demo:seed, uploads:prune

Mappa	Tartalom
app/Http/Controllers	Vezérlők: Pet, Shelter, Staffing, Form, Settings, Welcome, valamint a hitelesítés vezérlői
app/Http/Middleware	EnsureUserHasRole, a szerepkör-ellenőrzés
app/Http/Requests	Űrlapvalidációs osztályok
app/Livewire	RegisterForm, ImageUploader, PetFilter, PetSpeciesBreedSelect
app/Models	User, Shelter, Pet, Specie, Breed, Form, Like, Comment, Adoption
app/Policies	ShelterPolicy
app/Support	UploadDisk, a feltöltött képek tárolásának egy helyre gyűjtött logikája
database/migrations, seeders, factories	Táblák, alap- és demo adatok, tesztadat-gyárak
resources/views	Blade nézetek és komponensek
routes	Webes, hitelesítési és konzolos (ütemezett) útvonalak
tests/Unit, tests/Feature	A Pest tesztek

3. táblázat: A Laravel alkalmazás fontosabb mappái

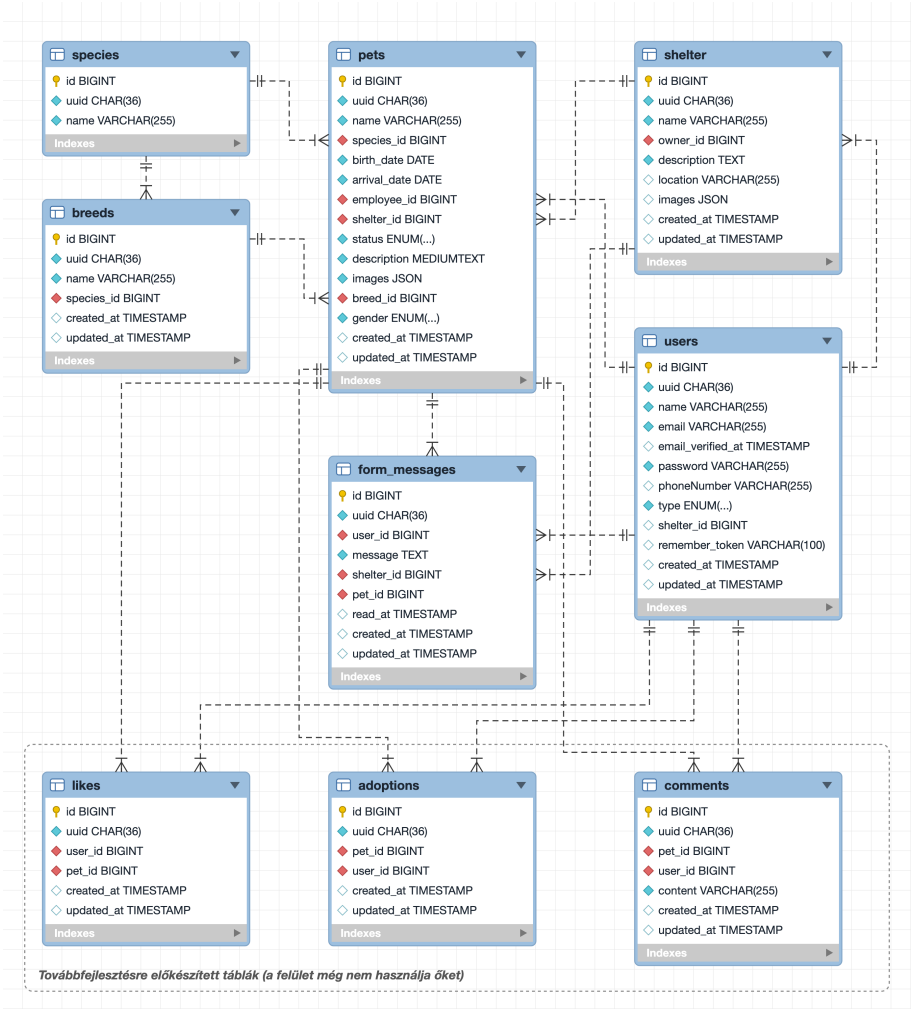
3.3 Adatmodell

Az adatbázis szerkezetét Laravel migrációkkal írtam le, így a php artisan migrate parancs bármelyik környezetben ugyanazt az adatbázist építi fel [19].

Tábla	Tartalom	Kapcsolatok
users	Felhasználói fiókok; a type mező a szerepkör	A munkatárs shelter_id mezője a menhelyére mutat

Tábla	Tartalom	Kapcsolatok
shelter	Menhelyek: név, leírás, település, képek (JSON)	owner_id: a tulajdonos
species, breeds	Fajok és fajták	Egy fajhoz több fajta tartozik
pets	Kisállatok: név, születési és bekerülési dátum, nem, státusz, leírás, képek (JSON)	Menhely, faj, fajta, a felvevő felhasználó (employee_id)
form_messages	A menhelynek küldött üzenetek	Örökbefogadó, menhely, kisállat
likes, comments, adoptions	Kedvelések, hozzászólások, örökbefogadások (előkészített táblák, felület még nincs hozzájuk)	Örökbefogadó, kisállat

4. táblázat: Az adatbázis fő táblái



6. ábra: Az adatbázis egyed-kapcsolat diagramja (egyszerűsített, a teljes változat a Függelékben)

A kapcsolatokat Eloquent relációkkal adtam meg: egy menhelynek több kisállata van (hasMany), egy kisállat pedig egy menhelyhez, fajhoz és fajtához tartozik (belongsTo). A munkatárs a users tábla shelter_id mezőjén keresztül kapcsolódik a menhelyhez, ezért egyszerre csak egy menhelyen dolgozhat.

A kisállat neme és státusza (elérhető, foglalt, örökbefogadott) enum mező.

Azonosítók az URL-ekben. Minden fő tábla kapott egy uuid oszlopot, és az URL-ekben a sorszám helyett ezt használom, így az azonosítókat nem lehet kitalálni. Az UUID-t a HasUuid trait hozza létre.

Kaszkádolt törlés. Ha egy tulajdonos törli a fiókját, vele együtt törlődik a menhelye, a kisállatai és az üzenetei is. A munkatárs fiókja viszont megmarad, csak a menhelyhez rendelése szűnik meg.

Képek tárolása. A képek elérési útját JSON tömbként tárolom a rekord images mezőjében.

3.4 Szerepkörök és jogosultságkezelés

Az alkalmazásban négy szerepkör van:

Szerepkör	Adatbázis-érték	Jogosultság
Látogató	nincs fiók	Böngészés
Örökbefogadó	User	Böngészés és üzenetküldés a menhelynek
Menhelytulajdonos	Shelterowner	Saját menhely, kisállatok, munkatársak és üzenetek kezelése
Munkatárs	Shelterworker	A menhelye kisállatainak kezelése

5. táblázat: Szerepkörök és jogosultságok

A jogosultságokat három szinten ellenőrzöm.

1. Bejelentkezés. A Laravel auth middleware-je a csak bejelentkezve elérhető oldalakról a bejelentkezési oldalra irányít.

2. Szerepkör. Az általam írt EnsureUserHasRole middleware azt ellenőrzi, hogy a felhasználó szerepköre szerepel-e az útvonalhoz megadott listában, ha nem, 403-as hibát ad. A kisállatkezelés például a tulajdonosnak és a munkatársnak érhető el, az üzenetküldés csak az örökbefogadónak.

3. Konkrét menhely. Egy tulajdonos csak a saját menhelyét szerkesztheti, egy munkatárs pedig csak a saját menhelye kisállatait kezelheti. Ezt a ShelterPolicy osztályban oldottam meg a Laravel policy rendszerével [20]:

```
class ShelterPolicy
{
    public function update(User $user, Shelter $shelter): bool
```

```
    {
        return (int) $shelter->owner_id === (int) $user->id;
    }

    public function manageStaff(User $user, Shelter $shelter): bool
    {
        return (int) $shelter->owner_id === (int) $user->id;
    }

    public function managePets(User $user, Shelter $shelter): bool
    {
        if ((int) $shelter->owner_id === (int) $user->id) {
            return true;
        }

        $worksAt = $user->worksAt;

        if (! $worksAt) {
            return false;
        }

        return (int) $worksAt->id === (int) $shelter->id;
    }
}
```

1. kódrészlet: A menhelyhez kötött jogosultságok (app/Policies/ShelterPolicy.php)

A policy szabályait az útvonalakon a can middleware, a vezérlőkben az authorize metódus alkalmazza.

3.5 Livewire komponensek

A projektben négy Livewire komponenst használok, ott, ahol a felhasználónak azonnal kell visszajelzést kapnia. Az ImageUploadert a következő alfejezetben mutatom be.

Komponens	**Feladata**
RegisterForm	A regisztrációs űrlap élő e-mail-ellenőrzéssel
ImageUploader	Képfeltöltés előnézettel, darabszám- és méretkorláttal
PetFilter	Szűrőgombok a kisállatlistán

Komponens	Feladata
PetSpeciesBreedSelect	Egymástól függő faj- és fajtaválasztó

6. táblázat: A Livewire komponensek

3.6 Képfeltöltés

A képfeltöltést két lépésben oldottam meg: a kép először egy ideiglenes mappába kerül, és csak az űrlap sikeres mentésekor kerül át a végleges helyére. Erre azért volt szükség, mert a felhasználó kitöltés közben tölti fel a képeket, és bármikor félbehagyhatja az űrlapot.

Az ImageUploader a Livewire fájlfeltöltésére épül [21]. Ellenőrzi a képek számát, méretét és típusát (JPEG vagy PNG), elmenti őket az ideiglenes mappába, az elérési utakat pedig a munkamenetben tárolja.

Mentéskor a vezérlő az UploadDisk::moveTempImages metódussal helyezi át a képeket a végleges mappába [22]:

```
public static function moveTempImages(string $context, string $folder): array
{
    $paths = [];

    foreach (session()->pull($context . '_temp_images', []) as $tmpPath) {
        if (! self::get()->exists($tmpPath)) {
            continue;
        }

        $newPath = $folder . '/' . basename($tmpPath);
        self::get()->move($tmpPath, $newPath);
        $paths[] = $newPath;
    }

    return $paths;
}
```

2. kódrészlet: Az ideiglenes képek áthelyezése (app/Support/UploadDisk.php)

Az UploadDisk az egyetlen hely, ahol eldől, melyik tárolón vannak a képek, így ezt egy környezeti változóval át lehet állítani.

3.7 A fő funkciók a kódban

Többlépéses regisztráció

A regisztráció a Breeze által generált kódra épül, ezt bővítettem ki több lépésre. Először a felhasználó kiválasztja, hogy örökbefogadóként vagy egy menhely részéről regisztrál, menhely esetén pedig azt is, hogy tulajdonos vagy munkatárs. A RegisteredUserController minden lépésnél ellenőrzi a választást, és ha valaki kihagy egy lépést, visszaküldi oda. A fiókot a RegisterForm komponens hozza létre, a tulajdonos pedig utána rögtön a menhely létrehozásához kerül.

Keresés és szűrés

A kisállatlistát a PetController index metódusa állítja elő. A keresett szöveget a kisállat nevében és a menhely településében keresi, a faj, nem és státusz szűrők pedig egyszerű feltételként kerülnek a lekérdezésbe.

Munkatárskezelés

A tulajdonos e-mail cím alapján vehet fel munkatársat. A StaffingController előtte ellenőrzi, hogy a felhasználó létezik-e, nem saját maga-e, és nem munkatárs-e már. Ha minden rendben van, a felhasználó szerepköre munkatársra változik, a fiókja a menhelyhez kapcsolódik. A munkatársat a tulajdonos eltávolíthatja, de ő maga is kiléphet.

Üzenetküldés a menhelynek

A bejelentkezett örökbefogadó a kisállat adatlapjáról üzenhet a menhelynek. A FormController ellenőrzi az üzenet hosszát (10-2000 karakter), és azt, hogy ugyanarról a kisállatról ne küldjön többször. A tulajdonos az üzeneteket listában látja, és törölheti őket. Az olvasatlan üzenetekre a menüben piros pont és darabszám figyelmeztet, megnyitáskor az üzenet read_at mezője kitöltődik, és olvasottá válik.

Validáció és lokalizáció

Az űrlapokon beküldött adatokat a Laravel validációs szabályaival ellenőrzöm [23], a bonyolultabb űrlapoknál (például a kisállat felvételénél) külön Form Request osztályban. A

felület magyar nyelvű, a Laravel beépített szövegeinek és hibaüzeneteinek magyar fordítása a lang/hu mappában és a hu.json fájlban van [24].

3.8 Ütemezett feladatok

Két feladat fut ütemezve a Laravel ütemezőjével, amelyet a szerveren egy percenként futó crontab bejegyzés indít [25].

Parancs	Időpont	Feladat
uploads:prune	naponta éjfélkor	A 24 óránál régebbi ideiglenes feltöltések törlése
demo:reset --force	naponta 04:00-kor	Minden adat törlése és a seedelt kiinduló állapot visszaállítása

7. táblázat: Ütemezett feladatok

3.9 Tesztelés

A tesztelést két szinten oldottam meg.

Szint	Eszköz	Mit ellenőriz	Darab
Unit és feature	Pest	Modellek, policy, middleware, validáció, vezérlők, útvonalak, Livewire komponensek, parancsok	272 teszt
E2E	Playwright	Teljes felhasználói folyamatok valódi böngészőben (Chromium, Firefox)	23 forgatókönyv, 46 futás

8. táblázat: A tesztelés szintjei

Unit és feature tesztek

A unit tesztek egy-egy osztályt vizsgálnak önmagában (modellek, policy, middleware, validáció), a feature tesztek pedig teljes HTTP kéréseket küldenek, és a választ ellenőrzik. A tesztek egy külön MySQL tesztadatbázist (tappmancs_test) használnak, és minden teszt üres állapotból indul.

Egy példa a ShelterPolicy tesztjei közül:

```
it('owner can manage staff', function () {
    $owner   = User::factory()->make(['id' => 1]);
    $shelter = Shelter::factory()->make(['owner_id' => 1]);

    expect((new ShelterPolicy())->manageStaff($owner, $shelter))->toBeTrue();
});

it('manageStaff casts ids to int before comparing', function () {
    $user    = User::factory()->make(['id' => 42]);
    $shelter = Shelter::factory()->make(['owner_id' => '42']);

    expect((new ShelterPolicy())->manageStaff($user, $shelter))->toBeTrue();
});
```

3. kódrészlet: Unit tesztek a jogosultságkezeléshez (tests/Unit/Policies)

Végponttól végpontig tartó (e2e) tesztek

Az e2e tesztek egy valódi böngészőt vezérelnek, és úgy használják az oldalt, ahogy egy felhasználó: kattintanak, kitöltik az űrlapokat, és ellenőrzik az eredményt. A tesztkódot három rétegre bontottam:

- **Spec fájlok:** a forgatókönyvek, területenként.
- **Lépések (steps):** üzleti szintű lépések, például „vegyél fel munkatársat”.
- **Page objectek:** az oldalak szelektorai, így ha a felület változik, egy helyen kell javítani [26].

```
test.describe('Staffing management @ui @staffing', () => {
    test('should let an owner add a worker and then remove them',
        async ({ registrationSteps, authSteps, staffingSteps, createUser }) => {
          const newWorker = createUser(USER_PREFIXES.staff);

          await registrationSteps.registerAdopter(newWorker);
          await authSteps.logout();
          await authSteps.login(env.shelterOwner.email, env.shelterOwner.password);
          await staffingSteps.openStaffingFromNavbar();
          await staffingSteps.addWorker(newWorker.email);
          await staffingSteps.removeWorker(newWorker.email);
        });
});
```

4. kódrészlet: Egy e2e forgatókönyv (app-e2e/src/ui/features/staffing/staffing.spec.ts)

A teszteket a helyi és az éles oldal ellen is le lehet futtatni, az éles oldalon csak azok futnak, amelyek nem módosítanak adatot. A tesztek a létrehozott fiókokat a végén törlik, ami mégis megmarad, azt a hajnali demo:reset takarítja el.

A legutóbbi e2e tesztfutás eredménye az oldal „Tudj meg többet” oldalán is látható.

3.10 Kihelyezés és üzemeltetés

Az éles szerver

Tulajdonság	Érték
Szolgáltató	DigitalOcean Droplet, Frankfurt
Méret	1 vCPU, 1 GB RAM, 25 GB SSD, 2 GB swap
Operációs rendszer	Ubuntu 24.04 LTS
Webszerver	nginx, PHP-FPM 8.5
Adatbázis	MySQL 8.0, ugyanazon a gépen
Képek	A gép lemezén (storage/app/public)
TLS	Let's Encrypt tanúsítvány (certbot), automatikus megújítással
Domain	tappmancs-szakdolgozat.hu

9. táblázat: Az éles szerver adatai

Az nginx a statikus fájlokat közvetlenül szolgálja ki, a PHP kéréseket a PHP-FPM-nek adja tovább, a HTTPS tanúsítványt pedig a certbot újítja meg automatikusan [27]. A tűzfal csak az SSH, HTTP és HTTPS portot engedi át, az adatbázist SSH-alagúton érem el.

A hosting kiválasztása

Az alkalmazás először AWS EC2-n futott SQLite-tal. Ezután kipróbáltam a Vercel serverless platformját, de ott a fájlrendszer csak olvasható, ezért az adatbázist és a képeket külön szolgáltatóknál kellett tárolni.

Végül egy DigitalOcean Droplet mellett döntöttem fix havidíjjal, ahol minden egy helyen fut.

Automatikus kihelyezés

A kihelyezést a GitHub Actions végzi, amikor a main ágra új változás kerül. A szakaszok egymás után futnak, és mindegyik csak akkor indul, ha az előző sikeres volt:

Szakasz	Mit csinál
Build assets	A GitHub szerverén (runner) lefordítja a CSS-t és a JavaScriptet
Run tests	Lefuttatja a backend teszteket egy MySQL tesztadatbázison; hiba esetén megáll
Pull code	SSH-n frissíti a kódot a szerveren, és telepíti a PHP függőségeket
Upload assets	Feltölti a lefordított fájlokat a szerverre
Migrate and cache	Adatbázis-migráció, a cache-ek újraépítése, a PHP-FPM újratöltése
Seed demo data	Csak kézi indításnál fut, ha a seed beállítás be van kapcsolva

10. táblázat: A kihelyezési folyamat szakaszai



7. ábra: A kihelyezési folyamat szakaszai a GitHub Actions felületén

A felület fájljait a GitHub gépén fordítom, mert a Vite fordítás kifuthatna a szerver 1 GB memóriájából. A backend tesztek kapuként működnek: ha egy is hibás, a kihelyezés el sem indul. Az e2e tesztek nincsenek a folyamatban, mert a már kihelyezett oldal ellen futnak.

3.11 AI-val gyorsított fejlesztés

A projektet AI-val gyorsított fejlesztéssel készítettem: a Claude Code segített a kódírásban, a tesztekben, a hibakeresésben, a szerver beállításában és a dokumentációban.

A tapasztalatom alapján ehhez három dolog kell:

- **Pontos feladatmegadás.** Minél pontosabban adom meg a feladatot és a kapcsolódó fájlokat, annál jobb az eredmény.
- **Ellenőrzés.** Az AI által írt kód csak akkor került be, ha a tesztek és a kódstílus-ellenőrzés hiba nélkül lefutott, és én is átnéztem.
- **Felelősség.** A tervezési döntéseket (architektúra, adatmodell, jogosultságok, hosting) én hoztam meg, az AI ezek megvalósítását gyorsította fel.

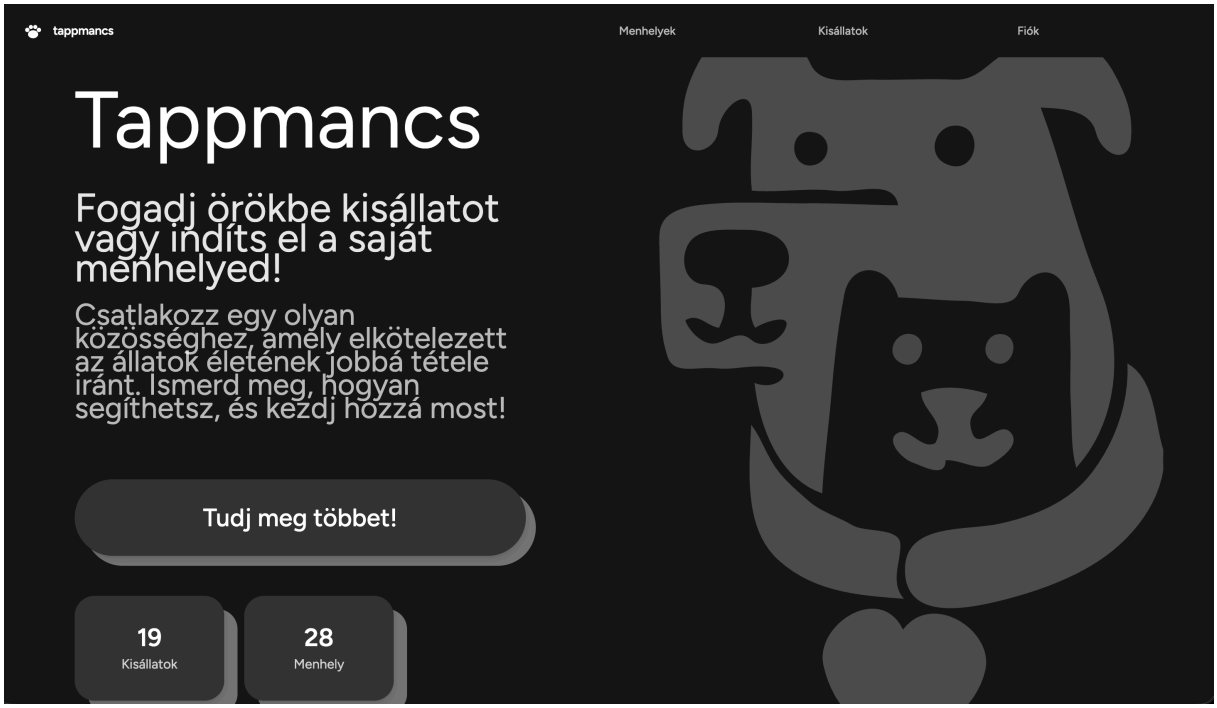
Így rövidebb idő alatt jutottam el egy tesztelt, élesben futó alkalmazásig, és több időm maradt a tervezésre és az ellenőrzésre.

B rész: Frontend (felhasználói útmutató)

Ebben a részben a felület használatát mutatom be. Az oldal a https://tappmancs-szakdolgozat.hu címen érhető el, a tesztfiókok a Függelékben vannak.

3.12 Böngészés látogatóként

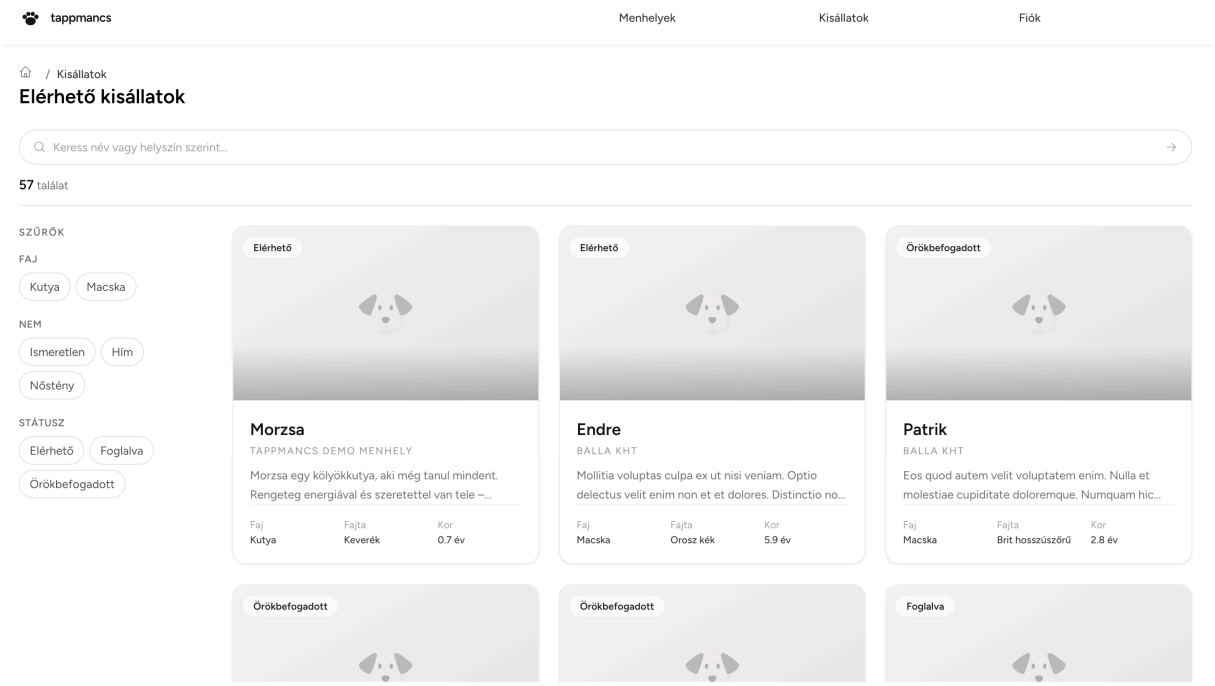
A nyitóoldal a legújabb kisállatokat és menhelyeket mutatja. A felső menüsorból érhető el a Kisállatok és a Menhelyek oldal, valamint a Bejelentkezés és a Regisztráció. A böngészéshez nem kell fiók.



8. ábra: A nyitóoldal

A Kisállatok oldalon kártyák mutatják az állatokat fényképpel, névvel, fajjal, fajtával, korral és menhellyel, a kép sarkában pedig egy címke jelzi a státuszt (elérhető, foglalt vagy örökbefogadott).

A lista felett a kisállat nevére vagy a menhely településére lehet keresni, és fajra, nemre és státuszra szűrni. A szűrők kombinálhatók, lapozásnál megmaradnak, és a szűrt lista linkje megosztható.

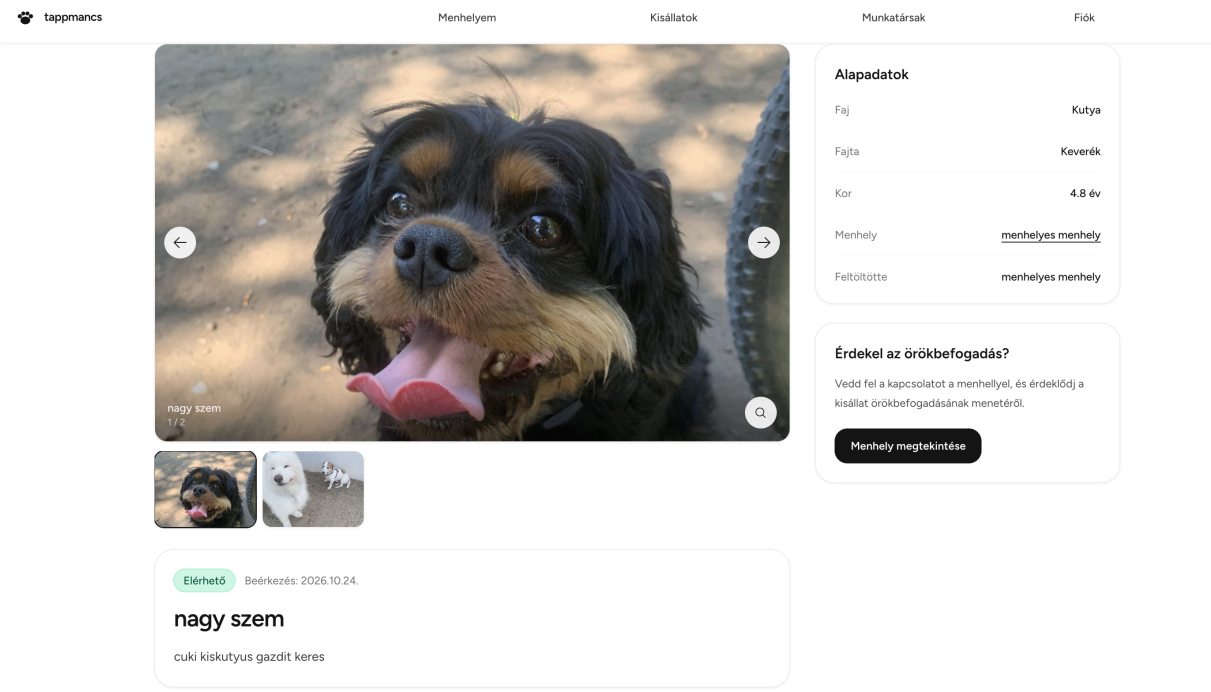


9. ábra: A kisállatlista keresővel és szűrőkkel

A Menhelyek oldal ugyanígy működik, itt a menhelyekre lehet név és település szerint keresni.

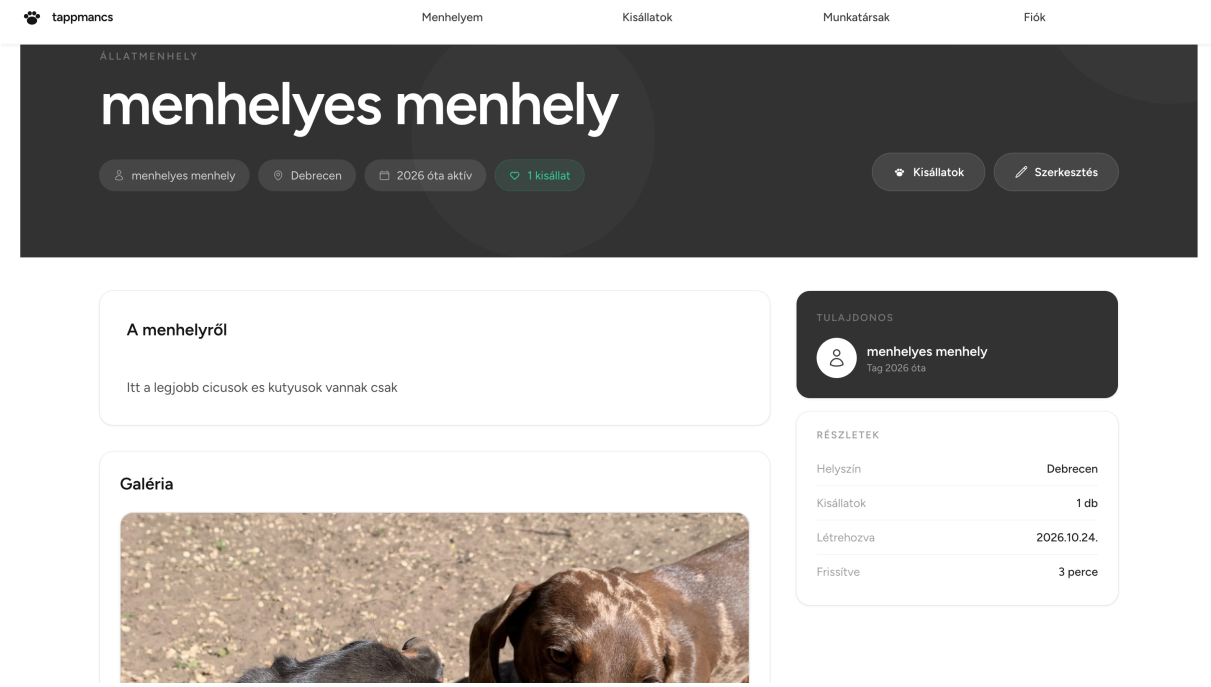
3.13 Kisállat- és menhelyadatlap

Egy kártyára kattintva megnyílik a kisállat adatlapja: felül egy nagyítható képgaléria, alatta az alapadatok és a leírás, az oldal alján pedig a menhely további kisállatai.



10. ábra: Egy kisállat adatlapja képgalériával

A menhely adatlapján a menhely leírása, települése és képei, valamint a menhely összes kisállata látható.

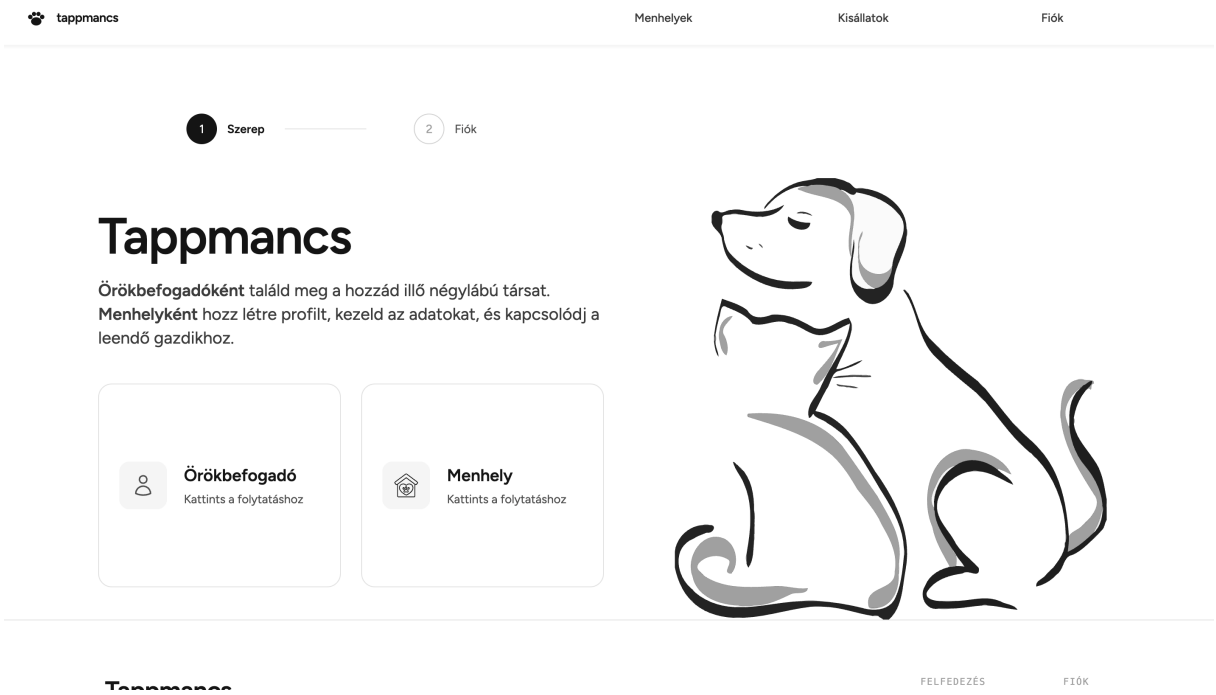


11. ábra: Egy menhely adatlapja

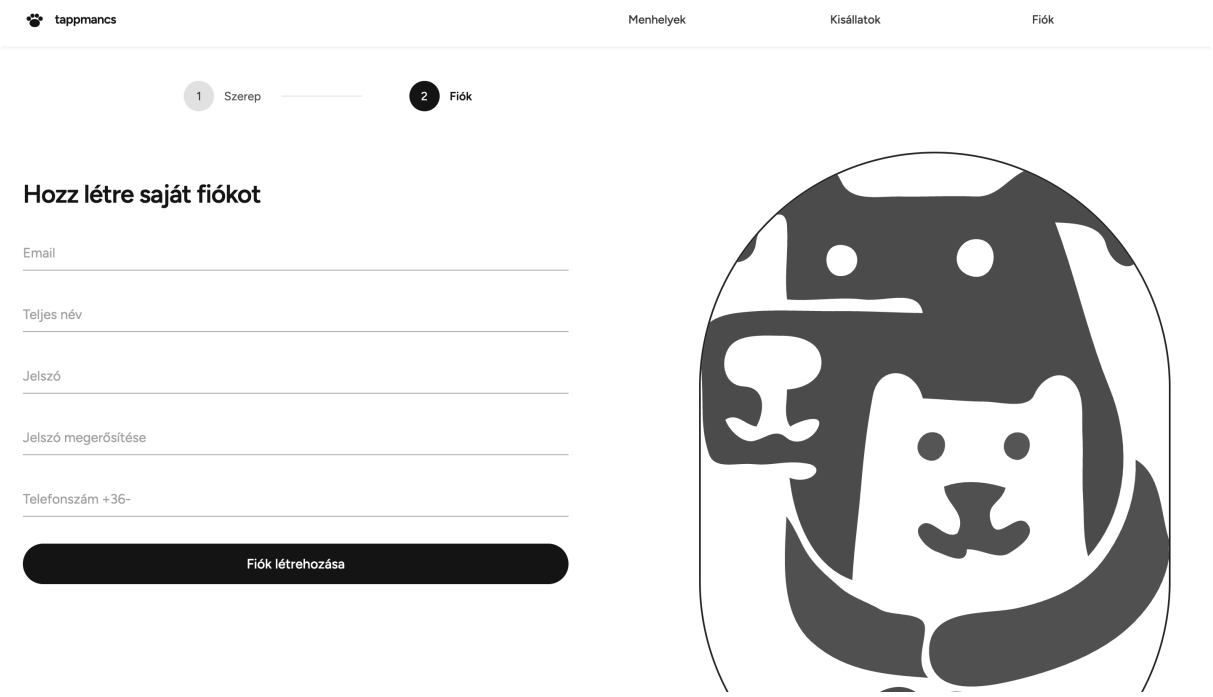
3.14 Regisztráció és bejelentkezés

A regisztráció lépései:

1. Először ki kell választani, hogy a felhasználó örökbefogadóként vagy menhelyként csatlakozik.
2. Menhely esetén azt is, hogy tulajdonosként vagy munkatársként.
3. Ezután a nevet, az e-mail címet, a jelszót és opcionálisan a telefonszámot kell megadni. A foglalt e-mail címet az oldal azonnal jelzi.
4. Tulajdonosként a regisztráció után a menhely létrehozása következik.



12. ábra: A regisztráció első lépése: szerepkörválasztás

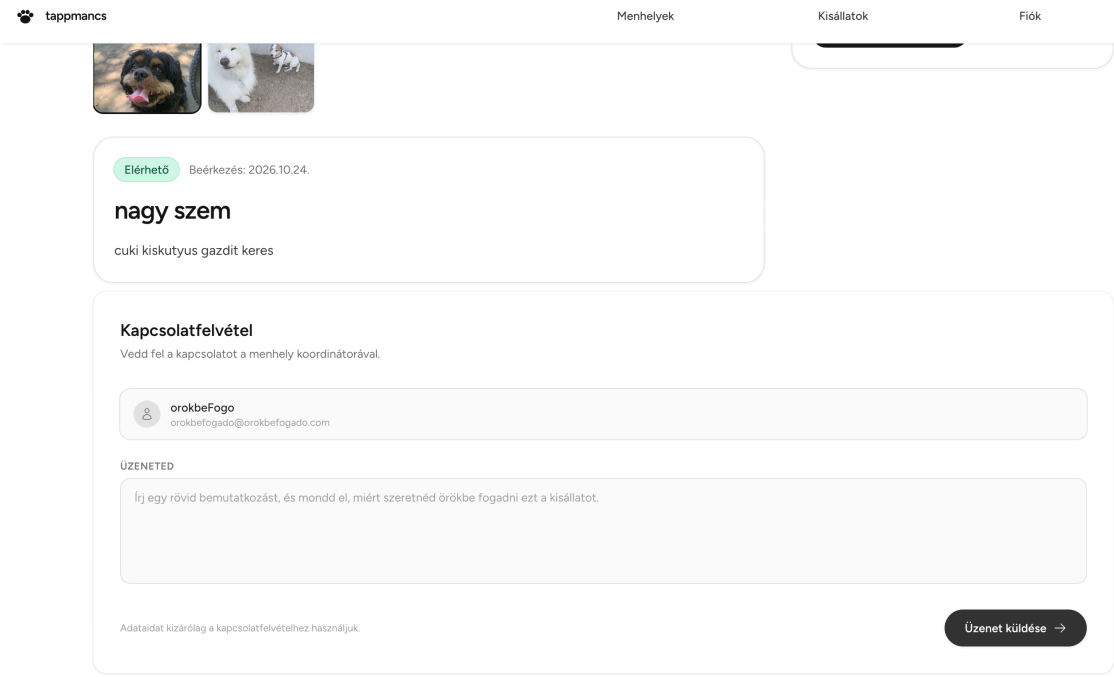


13. ábra: A regisztrációs űrlap

Bejelentkezni e-mail címmel és jelszóval lehet, elfelejtett jelszó esetén a bejelentkezési oldalon kérhető visszaállító link.

3.15 Örökbefogadóként: üzenet a menhelynek

Örökbefogadóként bejelentkezve a kisállat adatlapján rövid üzenetet lehet küldeni a menhelynek. Ugyanarról a kisállatról csak egy üzenet küldhető.

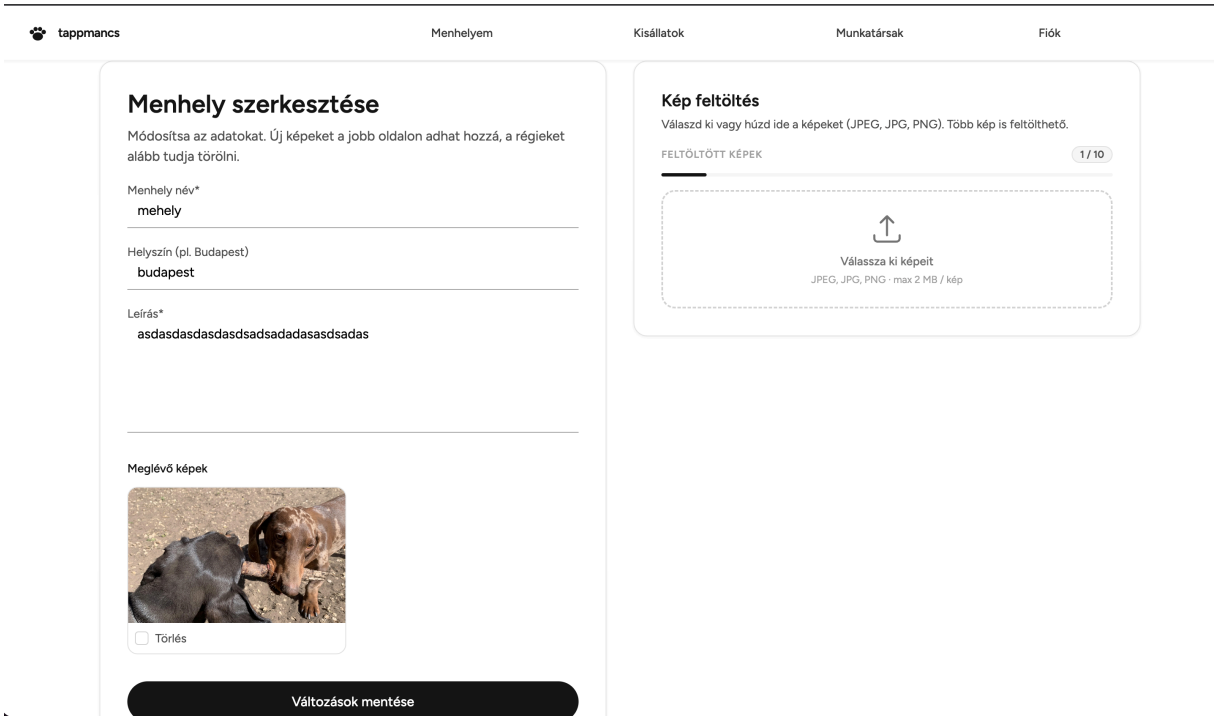


14. ábra: Az üzenetküldő űrlap a kisállat adatlapján

3.16 Menhelytulajdonosként

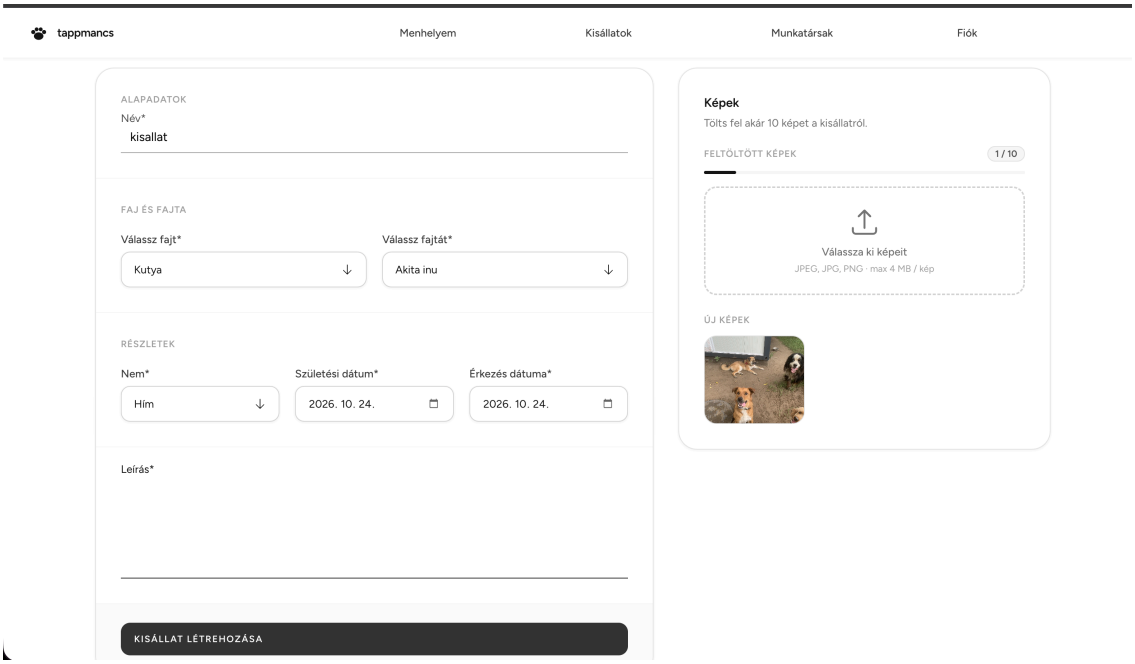
A menhelytulajdonos a saját funkcióit a menüsorban, a Kezelés részben éri el.

A menhely szerkesztése. A Szerkesztés menüpontban módosítható a menhely neve, leírása, települése és a képei.



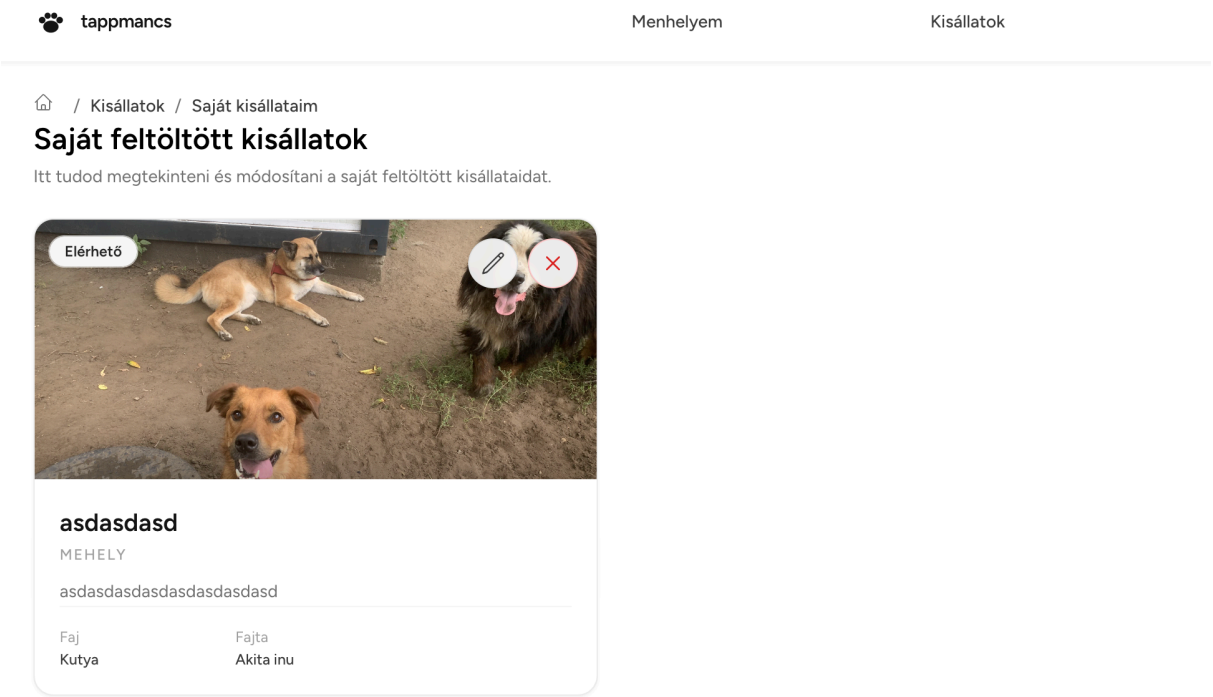
15. ábra: A menhely szerkesztése

Új kisállat felvétele. Az Új kisállat menüpontban kell megadni a kisállat adatait és feltölteni a képeit (legfeljebb 2 MB, JPEG vagy PNG). A fajták listája a kiválasztott fajhoz igazodik.



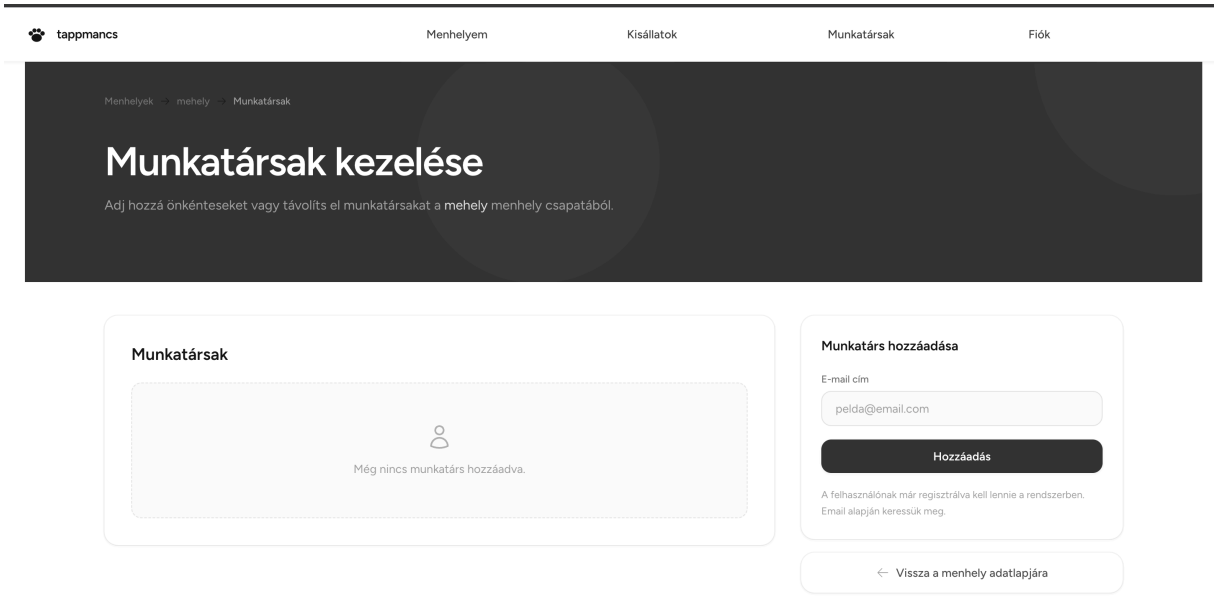
16. ábra: Az új kisállat felvételének űrlapja képfeltöltéssel

A kisállatok kezelése. A Saját kisállatok oldalon lehet a kisállatokat szerkeszteni, törölni, és gyorsan átállítani a státuszukat.



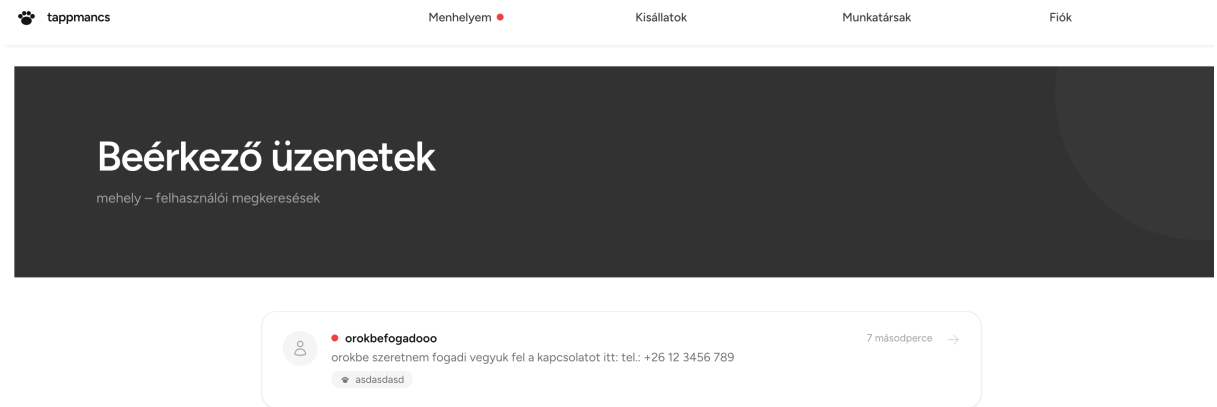
17. ábra: A saját kisállatok listája

Munkatársak. Itt e-mail cím alapján lehet munkatársat felvenni vagy eltávolítani. A felvett munkatárs onnantól kezelheti a menhely kisállatait.



18. ábra: A munkatársak kezelése

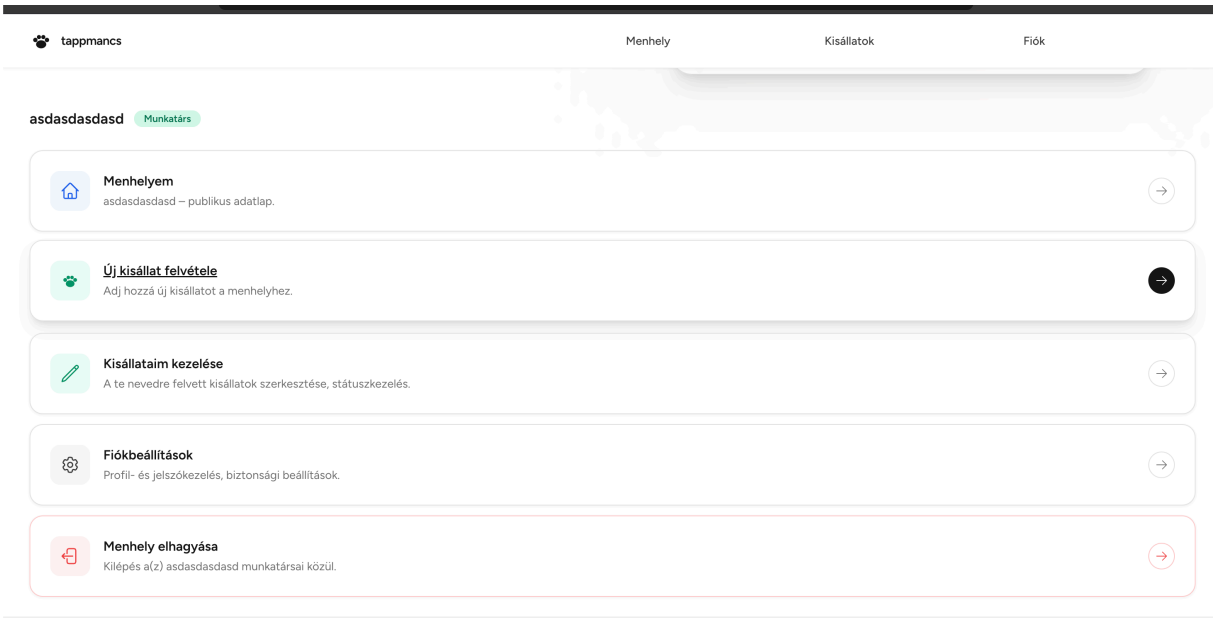
Üzenetek. Itt láthatók az örökbefogadóktól érkezett üzenetek, amelyeket meg lehet nyitni és törölni. Az olvasatlanok számát a menü az Üzenetek mellett mutatja.



19. ábra: A beérkezett üzenetek listája

3.17 Munkatársként

A munkatárs a regisztráció után még nem tartozik menhelyhez. Amikor egy tulajdonos felveszi, onnantól ugyanúgy kezelheti a kisállatokat, mint a tulajdonos, a menhely adatait és a munkatársakat viszont nem. A vezérlőpulton a Menhely elhagyása gombbal ki is léphet.

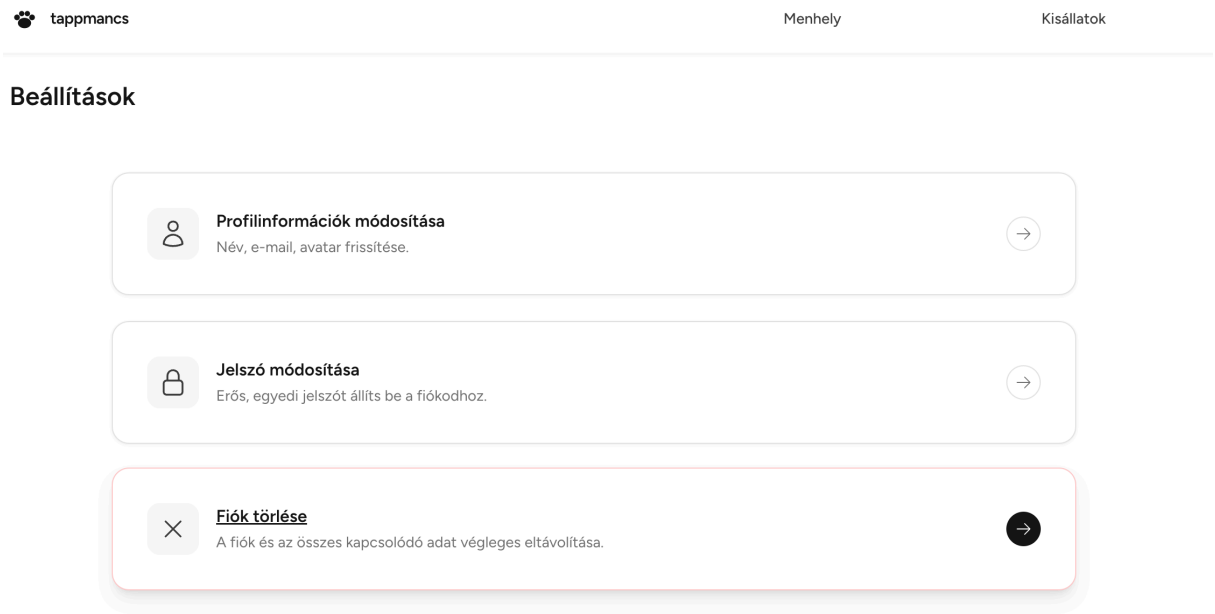


20. ábra: A munkatárs vezérlőpultja

3.18 Fiókbeállítások

A Beállítások oldalon minden bejelentkezett felhasználó:

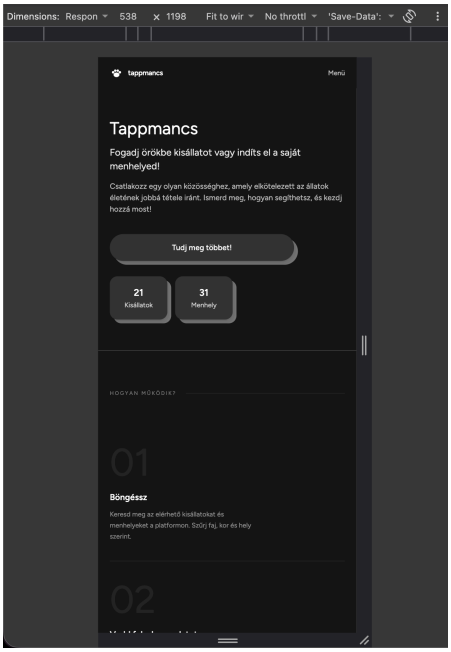
- módosíthatja a profiladatait (név, e-mail cím, telefonszám),
- megváltoztathatja a jelszavát,
- törölheti a fiókját (tulajdonosnál ezzel a menhely és a kisállatok is törlődnek).



21. ábra: A fiókbeállítások oldala

3.19 Mobilnézet

Az oldal reszponzív, kis képernyőn a menüt egy gombbal lehet lenyitni, a kártyák pedig egymás alá rendeződnek.



22. ábra: Az oldal mobilnézetben

4. Összefoglalás

4.1 Az eredmények

Elkészült a Tappmancs, egy élesben futó webalkalmazás, amely egy helyen köti össze a menhelyeket az örökbefogadókkal. Az 1.3. alfejezet céljai közül megvalósult a böngészés kereséssel és szűréssel, a négy szerepkör, a többlépéses regisztráció, a kisállatok és a menhelyek kezelése, a munkatárskezelés és az üzenetküldés.

A működést 272 backend és 23 e2e teszt ellenőrzi, és az oldal minden változás után automatikusan frissül, de csak akkor, ha minden backend teszt sikeres.

4.2 Mit tanultam a projektből?

- Egy alkalmazás nem ér véget a funkciók megírásával: az üzemeltetés (szerver, adatbázis, tanúsítvány, mentés, kihelyezés) legalább akkora munka.
- Az automatizált tesztek nemcsak hibát találnak, hanem bátorságot adnak a nagyobb átalakításokhoz, például a hosting vagy az adatbázis cseréjéhez.
- Az AI asszisztens hatékony eszköz, de csak pontos feladatmegadással és alapos ellenőrzéssel ad jó eredményt.

4.3 Új technológiák és módszerek

A Laravel és a Livewire keretrendszerként teljesen új volt számomra. A PHP-t viszont korábban már több projektben használtam, ezért a Laravel felépítését könnyű volt átlátni, és gyorsan elsajátítottam, amit a keretrendszer használatához tudni kell.

4.4 Mi volt nehéz és mi volt egyszerű?

Rész	Nehézség	Miért
Hosting és kihelyezés	Nehéz	Több platformot is ki kellett próbálni, mire a végleges megoldás összeállt; a szerver beállítása sok apró lépésből áll
Képfeltöltés	Nehéz	Az ideiglenes és a végleges tárolás, a korlátok és a félbehagyott feltöltések kezelése sok esetet jelent
Jogosultságkezelés	Közepes	A három réteg (bejelentkezés, szerepkör, konkrét menhely) átgondolást igényelt
Listák, keresés, szűrés	Egyszerű	A Laravel lekérdezésépítője és lapozása ezt jól támogatja

11. táblázat: A projekt nehezebb és egyszerűbb részei

4.5 Meg nem valósult célkitűzések

A tervből három funkció nem készült el: az adminisztrátori felület és a moderálás, a kommentelés és kedvelés, valamint az örökbefogadási folyamat. Az utóbbi kettő táblái már léteznek, de felület még nem épül rájuk. Fontosabbnak tartottam, hogy a fő folyamatok és a munkatárskezelés teljesen és tesztelten elkészüljenek.

4.6 Továbbfejlesztési lehetőségek

- Kedvelések, hozzászólások és örökbefogadási folyamat a meglévő táblákra építve.
- Több menhely kezelése egy tulajdonossal.
- Adminisztrátori szerepkör (a Developer típus az adatbázisban már létezik, jogosultság nélkül).
- Válasz az üzenetekre az oldalon belül, és értesítés új üzenetről.
- Kedvencek listája és értesítés, ha egy kedvenc kisállat státusza megváltozik.

Irodalomjegyzék

Internetes források

[1] Laravel, „Installation,” Laravel 11.x documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://laravel.com/docs/11.x/installation

[2] Livewire, „Quickstart,” Livewire documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://livewire.laravel.com/docs/quickstart

[3] Alpine.js, „Start Here,” Alpine.js documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://alpinejs.dev/start-here

[4] Laravel, „Blade Templates,” Laravel 11.x documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://laravel.com/docs/11.x/blade

[5] Laravel, „Starter Kits,” Laravel 11.x documentation. Hozzáférés: 2026. október 19. [Online]. Elérhető: https://laravel.com/docs/11.x/starter-kits

[6] Tailwind Labs, „Utility-First Fundamentals,” Tailwind CSS documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://tailwindcss.com/docs/utility-first

[7] Vite, „Getting Started,” Vite documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://vite.dev/guide/

[8] Oracle, „MySQL 8.0 Reference Manual,” MySQL documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://dev.mysql.com/doc/refman/8.0/en/

[9] Pest, „Installation,” Pest documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://pestphp.com/docs/installation

[10] Microsoft, „Installation,” Playwright documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://playwright.dev/docs/intro

[11] DigitalOcean, „Droplets,” DigitalOcean documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://docs.digitalocean.com/products/droplets/

[12] F5, „nginx documentation,” nginx.org. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://nginx.org/en/docs/

[13] GitHub, „Understanding GitHub Actions,” GitHub Docs. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://docs.github.com/en/actions/about-github-actions/understanding-github-actions

[14] Anthropic, „Claude Code overview,” Anthropic documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://docs.anthropic.com/en/docs/claude-code/overview

[15] npm, „workspaces,” npm Docs. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://docs.npmjs.com/cli/v10/using-npm/workspaces

[16] Laravel, „Routing,” Laravel 11.x documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://laravel.com/docs/11.x/routing

[17] Laravel, „Middleware,” Laravel 11.x documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://laravel.com/docs/11.x/middleware

[18] Laravel, „Eloquent: Relationships,” Laravel 11.x documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://laravel.com/docs/11.x/eloquent-relationships

[19] Laravel, „Database: Migrations,” Laravel 11.x documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://laravel.com/docs/11.x/migrations

[20] Laravel, „Authorization,” Laravel 11.x documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://laravel.com/docs/11.x/authorization

[21] Livewire, „File Uploads,” Livewire documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://livewire.laravel.com/docs/uploads

[22] Laravel, „File Storage,” Laravel 11.x documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://laravel.com/docs/11.x/filesystem

[23] Laravel, „Validation,” Laravel 11.x documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://laravel.com/docs/11.x/validation

[24] Laravel, „Localization,” Laravel 11.x documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://laravel.com/docs/11.x/localization

[25] Laravel, „Task Scheduling,” Laravel 11.x documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://laravel.com/docs/11.x/scheduling

[26] Microsoft, „Page object models,” Playwright documentation. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://playwright.dev/docs/pom

[27] Electronic Frontier Foundation, „Certbot documentation,” Certbot. Hozzáférés: 2026. október 18. [Online]. Elérhető: https://eff-certbot.readthedocs.io/en/stable/

Függelék

F.1 Tesztfiókok

Az alábbi fiókok a helyi és az éles oldalon is elérhetők, és a hajnali visszaállítás után is mindig megvannak.

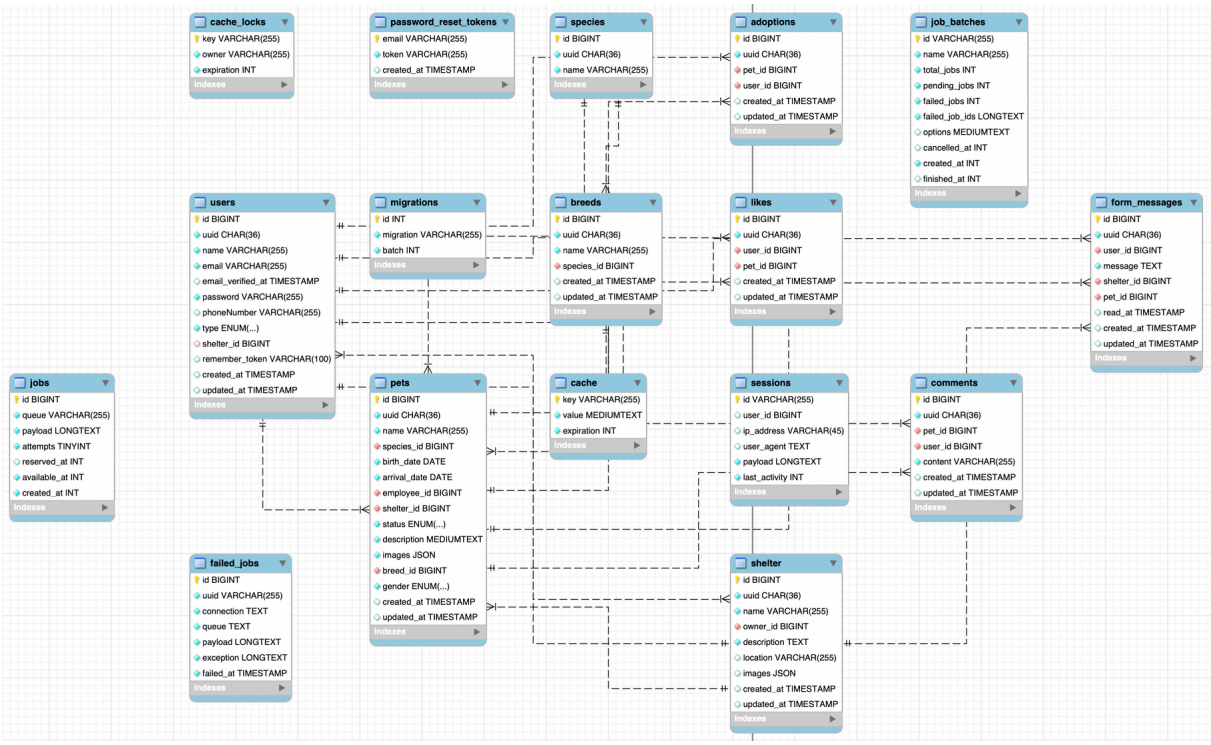
Szerepkör	E-mail	Jelszó
Örökbefogadó	user@example.com	RegularUserPassword123
Menhelytulajdonos	shelterowner@example.com	ShelterOwnerPassword123
Munkatárs	shelterworker@example.com	ShelterWorkerPassword123
Demo menhelytulajdonos	demo.owner@example.com	DemoPassword123
Demo munkatárs	demo.worker@example.com	DemoPassword123
Demo örökbefogadó	demo.user@example.com	DemoPassword123

12. táblázat: Tesztfiókok

A demo tulajdonos menhelye a „Tappmancs Demo Menhely” három kisállattal. A teljes seedelt adatállomány 28 felhasználót, 28 menhelyet és 57 kisállatot tartalmaz.

F.2 Az adatbázis teljes egyed-kapcsolat diagramja

A teljes diagram a Laravel saját tábláit is mutatja: sessions, password_reset_tokens, cache, cache_locks, jobs, job_batches és failed_jobs (az alap migrációkból), valamint a migrations táblát, amelyben a php artisan migrate a lefuttatott migrációkat tartja nyilván.



23. ábra: Az adatbázis teljes egyed-kapcsolat diagramja

F.3 A projekt időrendje

Időszak	Mi készült
2024. december	Laravel projekt, adatmodell, migrációk, Eloquent kapcsolatok
2025. február és március	Többlépéses regisztráció, nyitóoldal, bejelentkezés
2025. június és szeptember	Felhasználói oldalak, az e2e tesztkörnyezet alapjai, kódstílus-ellenőrzés
2026. március és április	Kisállatlista, adatlap, szerkesztés és törlés, menhelyadatlap galériával, monorepo, hibaoldalak
2026. május	Keresés és szűrés, munkatárskezelés
2026. június	Üzenetküldés, GitHub Actions kihelyezés, mobilnézet
2026. október	Saját szerver, domain és TLS, MySQL, napi demo-visszaállítás, e2e az éles oldal ellen

13. táblázat: A projekt időrendje

Köszönetnyilvánítás

Köszönöm témavezetőmnek, Major Sándor Rolandnak, hogy a fejlesztés során rendszeresen átnézte velem az elkészült részeket, segített a sprintek tervezésében, és tanácsaival támogatta a munkámat.